

Thinking Right About Problem Solving – Iterative vs. Recursive Approaches

Analysis and Design of Algorithms



Recursion naturally expresses the
"structure" of the problem



Problem 1: Count Digits



Problem 1: Count Digits

```
def count_digits(n):  
    count = 0  
    while n > 0:  
        count += 1  
        n //= 10  
    return count
```

Your task: Rewrite this recursively



Problem 1: Count Digits

```
def count_digits(n):  
    count = 0  
    while n > 0:  
        count += 1  
        n //= 10  
    return count
```

Your task: Rewrite this recursively

Hint: How can you break "count digits of n" into a simpler version?



Problem 1: Count Digits

```
def count_digits(n):  
    count = 0  
    while n > 0:  
        count += 1  
        n //= 10  
    return count
```

Your task: Rewrite this recursively

Hint: How can you break "count digits of n" into a simpler version?

Think: $\text{count_digits}(1234) = 1 + \text{count_digits}(123)$



Problem 1: Count Digits

```
def count_digits(n):  
    count = 0  
    while n > 0:  
        count += 1  
        n //= 10  
    return count
```

Recursive Solution

```
def count_digits_recursive(n):  
    if n < 10:  
        return 1 if n > 0 else 0  
  
    return 1 +  
        count_digits_recursive(n // 10)
```



Problem 1: Count Digits

```
def count_digits(n):  
    count = 0  
    while n > 0:  
        count += 1  
        n //= 10  
    return count
```

Example: `count_digits_recursive(1234)`
= 1 + `count_digits_recursive(123)`
= 1 + 1 + `count_digits_recursive(12)`
= 1 + 1 + 1 + `count_digits_recursive(1)`
= 1 + 1 + 1 + 1 = 4

Key thinking shift: Instead of "remove digits until done", think "this digit + count remaining digits."

Recursive Solution

```
def count_digits_recursive(n):  
    if n < 10:  
        return 1 if n > 0 else 0  
  
    return 1 + count_digits_recursive(n // 10)
```



Problem 2: Find Maximum Value

- [3,9,1,4,7]



Problem 2: Find Maximum Value

```
def find_max(arr):  
    max_val = arr[0]  
    for i in range(1, len(arr)):  
        if arr[i] > max_val:  
            max_val = arr[i]  
    return max_val
```

Your task: Rewrite this recursively



Problem 2: Find Maximum Value

```
def find_max(arr):  
    max_val = arr[0]  
    for i in range(1, len(arr)):  
        if arr[i] > max_val:  
            max_val = arr[i]  
    return max_val
```

Your task: Rewrite this recursively



Problem 2: Find Maximum Value

```
def find_max(arr):  
    max_val = arr[0]  
    for i in range(1, len(arr)):  
        if arr[i] > max_val:  
            max_val = arr[i]  
    return max_val
```

Your task: Rewrite this recursively

Think: Max of [3, 7, 2, 9, 1] = max(3, max([7, 2, 9, 1]))



Problem 2: Find Maximum Value

```
def find_max(arr):  
    max_val = arr[0]  
    for i in range(1, len(arr)):  
        if arr[i] > max_val:  
            max_val = arr[i]  
    return max_val
```

Recursive Solution

```
def find_max_recursive(arr):  
    if len(arr) == 1:  
        return arr[0]  
  
    max_of_rest = find_max_recursive(arr[1:])  
    return arr[0] if arr[0] > max_of_rest else  
        max_of_rest
```



Problem 3: Palindrome

A **palindrome** is a string that reads the same backward as forward, such as "madam" or "racecar".

Problem Statement:

Design an algorithm to find the **longest palindromic substring** within a given string S of length n .

Your algorithm should:

Ignore spaces and consider only **alphanumeric characters**.

Treat uppercase and lowercase letters as **the same** (case-insensitive).

Return both the **longest palindromic substring** and its **length**.



Problem 3: Palindrome

A **palindrome** is a string that reads the same backward as forward, such as "madam" or "racecar".

Problem Statement:

Design an algorithm to find the **longest palindromic substring** within a given string S of length n .

Your algorithm should:

Ignore spaces and consider only **alphanumeric characters**.

Treat uppercase and lowercase letters as **the same** (case-insensitive).

Return both the **longest palindromic substring** and its **length**.



Problem 3: Palindrome

```
def is_palindrome(s, left=0, right=None):  
    if right is None:  
        right = len(s) - 1  
    if left >= right:  
        return True  
    if s[left] != s[right]:  
        return False  
    return is_palindrome(s, left + 1, right - 1)
```



Problem 3: Palindrome

`is_palindrome("racecar")`

Checking: `s[0]='r'` vs `s[6]='r'`

Checking: `s[1]='a'` vs `s[5]='a'`

Checking: `s[2]='c'` vs `s[4]='c'`

Checking: `s[3]='e'` vs `s[3]='e'`

Base case reached → Palindrome



Problem 4: Climbing Stairs

A person is climbing a staircase with **$n + 1$ steps**, numbered from 0 to n . Each step i (where $1 \leq i \leq n$) has an associated **cost**, given in an array `costs`, where `costs[i]` represents the cost of step i .

- From any step i , the person can jump to **step $i + 1$** , **step $i + 2$** , or **step $i + 3$** .
The cost of jumping from step i to step j is calculated as:

$$\text{cost}(i \rightarrow j) = \text{costs}[j] + (j - i)^2$$

- The person starts from **step 0** with a total cost of **0**.

Problem Statement:

- Design a **recursive algorithm** to compute the **minimum total cost** required to reach step n .



Problem 4: Climbing Stairs

```
costs = [0, 5, 10, 3, 7]  
n= len(costs) - 1  
print(min_cost_to_reach(costs, n))
```



Problem 4: Climbing Stairs



Problem 4: Climbing Stairs

```
def min_cost_to_reach(costs, i):  
    if i == 0:  
        return 0  
    res = float('inf')  
    if i - 1 >= 0:  
        res = min(res, min_cost_to_reach(costs, i - 1) + costs[i] + (1 ** 2))  
    if i - 2 >= 0:  
        res = min(res, min_cost_to_reach(costs, i - 2) + costs[i] + (2 ** 2))  
    if i - 3 >= 0:  
        res = min(res, min_cost_to_reach(costs, i - 3) + costs[i] + (3 ** 2))  
    return res
```



Problem 5: Longest Substring Without Repeating Characters

```
print("Result =", length_of_longest_substring("abcabcbb"))
```



Problem 5: Longest Substring Without Repeating Characters

```
def longest_unique_substring(s, current="", index=0):
    if index == len(s):
        return len(current)
    if s[index] in current:
        next_index = current.index(s[index]) + 1
        return max(len(current), longest_unique_substring(s,
            current[next_index:] + s[index], index + 1))
    else:
        return longest_unique_substring(s, current + s[index], index + 1)
def length_of_longest_substring(s):
    return longest_unique_substring(s)
```



The Ultimate Question to Ask:
"Can I express the solution to this problem in
terms of solutions to smaller versions of the
same problem?"
If YES → Think recursively first!

